A Comparative Analysis of Broadcasting Protocols in Blockchain Networks

Yojit Kumar (20231289) | 3rd year BS-MS, IISER Pune | **Supervisor: Dr Kalpesh Kapoor** | DS3613 Semester Project

1. Epidemic Modelling & Gossip Protocols

My interest in this project originates from **epidemic modelling**, the study of how information, diseases, or disturbances propagate through a network of interacting agents. Gossip protocols borrow directly from this mathematics [2]. Nodes occupy three states:

Susceptible $\longrightarrow$ **Infected** $\longrightarrow$ **Removed**

(uninformed) (has block) (done forwarding)

The **SI model** corresponds to anti-entropy: every informed node continuously polls neighbours and pushes its data. The **SIR model** corresponds to rumour-mongering: a node stops spreading with probability $1/k$ after each round, modelling the decay of interest in already-known information [2].

In a gossip protocol, coverage grows exponentially at first then saturates; exactly the logistic curve of an epidemic. This gives us a principled way to study broadcast in decentralised systems: the same mathematical tools that model disease spread can bound how quickly information reaches all nodes.

2. Blockchains & Broadcast Protocols

A blockchain is a distributed, append-only ledger that records transactions across a peer-to-peer network **without relying on a central authority** [11]. Because every node maintains a complete replica, every new block mined must be propagated to *all* participants so that local copies remain synchronised [1].

The mechanism that achieves this is the **broadcast protocol** — the rules by which a message originating at one node is relayed until it reaches every other node [1]. In Bitcoin's original implementation, a node announces a new block via an INV message (block hash only); neighbours without the block reply with GETDATA and receive the full block. This push-pull cycle repeats hop-by-hop across the network.

Slow propagation is a security vulnerability. The median time for a block to reach all Bitcoin nodes is 6.5 s, with 5% still uninformed after 40 s. During this window a competing miner may produce a conflicting block, causing a fork (≈1.7% observed rate [1]). An attacker holding as little as 49.1% of hash power can mount a double-spend attack, a threshold that drops to ≈30% under the threat of Selfish Mining [3].

3. Measuring a Broadcast Protocol

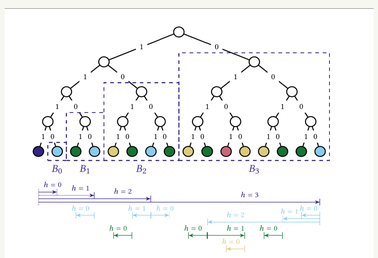
A broadcast protocol must satisfy competing requirements simultaneously [1, 6]. Three primary performance metrics are:

- **Propagation latency** — the time from block creation at the source to receipt at a target fraction of nodes (e.g. 50%, 90%, 99%). Directly determines fork rate and consensus security.
- **Bandwidth overhead** — the ratio of total messages transmitted across the network to the theoretical minimum needed for full coverage. Naive flooding achieves zero latency but at $\mathcal{O}(N \times d)$ message complexity [1].
- **Reliability & fault tolerance** — the fraction of nodes that successfully receive the block under node failures, packet loss, and adversarial behaviour. A protocol that is fast but fragile is worse than one that is slower but delivers with certainty.

These three axes are in tension: reducing latency typically increases overhead; improving fault tolerance often increases both. Understanding this tradeoff is the central theme of this project [6].

4. KadCast

- **Foundation:** Kadcast [13] is built on Kademlia [10], a DHT in which each node holds a random L-bit identifier and distance is the XOR metric: $d(x, y) = (x \oplus y)_{10}$.
- **Overlay Structure:** Each node organises known peers into k-buckets, where B_i holds nodes at XOR distance $[2^i, 2^{i+1})$, enabling $O(\log N)$ traversal.
- **Mechanism:** A source selects β nodes from each of its L buckets and sends a CHUNK message with broadcast height h. Each receiver repeats this for buckets below h, recursively delegating subtree responsibility.
- This implicit spanning tree covers all N nodes in $\Theta(\log N)$ steps with no central coordinator [13].



Reliability: (i) $\beta > 1$ delegates per bucket; adversarial probability $(M/N)^\beta$ falls exponentially. (ii) RaptorQ FEC: any s of $n > s$ encoded symbols suffice for reconstruction [13].

- **Performance:** In simulation ($N = 500$, 1 MB blocks), Kadcast reached 90% coverage in 3784 ms vs. 8461 ms for vanilla gossip over 50% faster [13].
- **Key limitation — subtree starvation:** The XOR-tree structure means broadcast responsibility is delegated recursively down subtrees. A single relay node at height h that fails or behaves maliciously can leave its *entire* subtree without the block [13].

5. Optimump2p/RLNC Gossip

- **Purpose:** Optimump2p [12] is designed as a replacement for Gossipsub in Ethereum 2.0's libp2p stack, built on RLNC [5]. Intermediate nodes forward **random linear combinations** of received packets over $\mathbb{F}_{2^q}$, it may achieve the min-cut capacity with error probability $\leq d/q$ [5].
- **Mechanism:** The publisher divides the block into k fragments, encodes into $p \cdot k$ coded shards, and pushes to all mesh neighbours. Each node accumulating $\geq r$ shards recodes and forwards. Any k linearly independent shards suffice for Gaussian elimination decoding.
- **Haeupler's proof** [4]: RLNC gossip achieves stopping time $\mathcal{O}(k + T)$ even in dynamically changing topologies.
- **Performace:** At 20 msg/s with 10 MB blocks: Optimump2p held 2 s latency and 99–100% delivery; Gossipsub degraded to 4 s and 80% delivery [12].
- **Key limitation — topology blindness:** Optimump2p does not exploit network structure to reduce hop count. The stopping time $\mathcal{O}(k + T)$ is optimal *given* T, but T itself grows with the network diameter or inversely with the conductance λ of the gossip mesh [4]. On a poorly connected or geographically dispersed network, T can be large, and RLNC does nothing to reduce it.

6. A Hybrid Proposition

- Combines Kadcast's structural routing with RLNC's algebraic resilience to to address orthogonal failure modes.
- Kadcast's XOR tree ensures $\Theta(\log N)$ hop depth, bounding T.
- RLNC eliminates subtree starvation by letting downstream nodes *pool partial coded shards* from multiple disrupted relay paths and collectively decode.
- **Design:** The source divides the block into k fragments and encodes them into n coded shards over $\mathbb{F}_{2^q}$ and selects one representative per Kademlia bucket as reciever.
- **Theoretical guarantee:** The broadcast executes on G_{kad}, with diameter $O(\log N)$ and isoperimetric number $h = \Omega(1/\log N)$. Applying Haeupler's Lemma 6.5 [4] gives single-message stopping time $T = O(\log^2 N)$; Theorem 4.3 (perfect pipelining) then extends this to:

$$T_{\text{KadRLNC}} = O\left(k + \log^2 N\right)$$

The latency advantage over Optimump2p holds when $D_{\text{gossip}} > \log N$.

7. Simulation Framework

Protocols are evaluated in a custom **discrete-event simulator** (SimPy) on a 1000-node Kademlia overlay with 16-bit identifiers. Routing tables are precomputed with full global knowledge, capping each bucket at 10 randomly sampled peers.

- **Bandwidth tiers:** 20% infrastructure nodes at 1 GB/s; 80% consumer nodes at 50 MB/s
- **Link latency:** sampled per-packet from LogNormal (median 50 ms)
- **Contention:** shard transmissions serialised through a per-node uplink queue (SimPy Resource, capacity 1)
- **RLNC overhead:** fixed 5 ms processing delay at recode and decode thresholds
- **Reproducible:** same graph instance and source node per protocol comparison; all stochastic elements seeded

8. Experimental Design

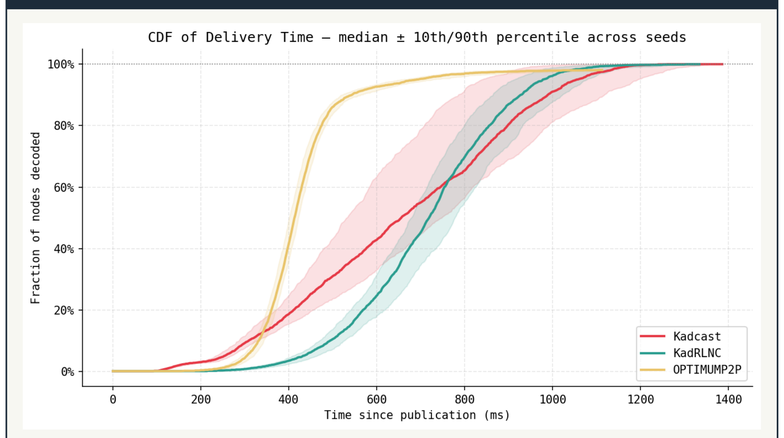
Experiment 1 — Baseline Dissemination. Single 1 MB block, no competing traffic. Five broadcasts × ten seeds = 50 measurements per protocol. Isolates pure latency and coverage.

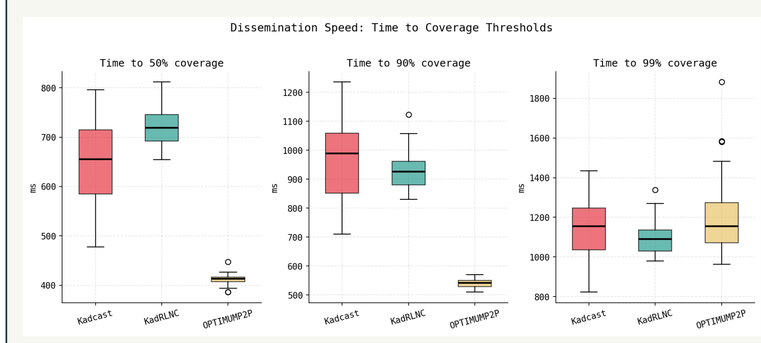
Experiment 2 — Publish Rate Stress Test. Injection rates $\lambda \in \{1, 2, 4, 8, 16, 32, 64\}$ msg/s on shared uplink queues. Blocks tracked over a 5 s window + 3 s cooldown to avoid queue-relaxation artefacts. Measures delivery rate and latency under contention.

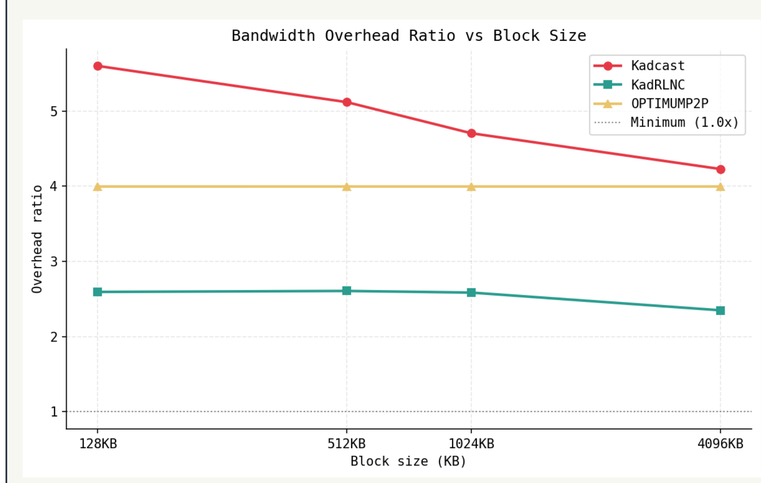
Experiment 3 — Payload Scaling. Block sizes $\in \{128\,\text{KB}, \ldots, 4\,\text{MB}\}$, $k = 32$ fixed (shard size scales proportionally). No contention. Isolates dissemination time and bandwidth overhead vs. payload.

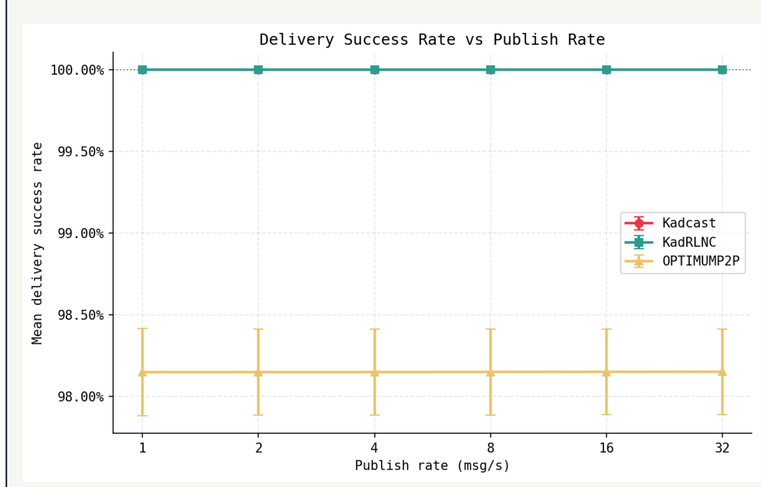
Metrics: Time-to-Coverage CDF (50%, 90%, 99%), Delivery Success Rate, Bandwidth Overhead (total shards / theoretical minimum).

9. Results









Kadcast OPTIMUMP2P KadRLNC

10. Conclusion & Future Directions

Kadcast exploits XOR geometry to achieve $\Theta(\log N)$ hop depth but its store-and-forward relay creates a hard subtree starvation vulnerability. Optimump2p achieves excellent delivery through RLNC-coded gossip but is topology-agnostic. KadRLNC was proposed to address both, with a theoretical stopping time of $O(k + \log^2 N)$.

The simulation results are more nuanced. OPTIMUMP2P was fastest at 50% and 90% coverage (median ≈410 ms vs ≈720 ms for KadRLNC). KadRLNC's genuine empirical advantage is **bandwidth efficiency** (≈2.5× overhead vs ≈4× for OPTIMUMP2P and ≈5.5× for Kadcast) and graceful payload scaling. The latency gap is likely a scale artefact: at $N = 1000$ with only 16 Kademlia buckets, the diameter advantage of G_{kad} is too small to overcome the fixed RLNC processing overhead.

Future directions:

- Evaluate KadRLNC at $N \geq 10{,}000$ where $D_{\text{gossip}} \gg \log N$ is the realistic regime
- Introduce dynamic Byzantine node models
- Formally validate the combined bandwidth-latency parameter against established metrics

References

[1] Decker & Wattenhofer. Information Propagation in the Bitcoin Network. *IEEE P2P*, 2013. [2] Demers et al. Epidemic Algorithms for Replicated Database Maintenance. *PODC*, 1987.
[3] Eyal & Sirer. Majority is not Enough. 2013. [4] Haeupler. Analyzing Network Coding Gossip Made Easy. *STOC*, 2010. [5] Ho et al. A Random Linear Network Coding Approach to Multicast. *IEEE Trans. Inf. Theory*, 2006.
[6] Lai et al. Accelerating Block and Transaction Propagation. *IEEE TNSE*, 2025. [7] Nakamoto. Bitcoin: A Peer-to-Peer Electronic Cash System. 2008.
[8] Nicolaou et al. OPTIMUMP2P: Fast and Reliable Gossiping in P2P Networks. 2025. [9] Rohrer & Tschorsch. Kadcast. *ACM AFT*, 2019.